NOVA-KV — Part D: Benchmarking Everything Together (corrected campaign)

Campaign `20261013T115839-fcd2938f49b6-all` (directory `results/part_d/unwrapped-20261013T115839-fcd2938f49b6/`), measured at commit `fcd2938`. Every number below is the **median of 5 independent processes, with [min–max]** where given. This file is generated from `reports/part_d_report.template.md` by `scripts/render_part_d_report.py`: each number is computed from the campaign's summary CSVs, which `scripts/aggregate_part_d.py` derives from the raw runs; none is typed in. Nothing was estimated and no run was dropped. 470 timing runs and 154 profiled runs, 0 failures, 0 problems found by the completeness check.

0. Why this is the third measurement, and what changed

This report replaces the one tagged `part-d-final`. Two errors of method were found in the first campaign, the second of them only because the first was being corrected.

1. The timed runs were inside `perf stat`. The first campaign (`20261013T090846-615f4357f015-all`) wrapped every timed run in `perf stat` so that counters and timings would come from the same process. That changes the timing where threads block often. Here the timed executables are **started directly**, by nothing but the shell, and all profiling is done afterwards in separate runs whose throughput is never used as a result (§1, §5). Measured in this campaign, same cell, same lock placement, with and without the wrapper: global mutex at 8 readers 1.25× (paced writer); shared mutex with an unthrottled writer and 1 to 4 readers 0.48–0.57×; stop-the-world 0.71× (workload A) and 0.41× (workload B); RCU at 8 readers 0.99× and 1.04×.

2. The position of the lock inside its cache line was not controlled, and it decides the speed of the lock-based stores. The second campaign (`20261013T105136-146b851dc3dd-all`, unwrapped) gave 34.17 M reads/s for the shared mutex at 8 readers where the first had given 21.37 M. The cause was not the wrapper. The lock object is the first member of the store; where it lands in a 64-byte cache line depends on the heap, and the heap depends on the lengths of the strings on the command line. Changing only the length of a label moves the lock, and the store runs at one of two speeds (`results/part_d_diagnostics/lock_placement_effect.txt`: 18 of 18 runs, and 16 of 16 with explicit placement, follow the rule). In the first two campaigns each cell had its own accidental offset (in that cell: 32 in the first campaign, 48 in the second), so their lock-store curves mix the two speeds. In this campaign the benchmark places the lock explicitly and **every lock-store cell is measured at offset 48 and at offset 0** (§1). The store's code is unchanged.

What changed in the conclusions.

Conclusion of the first report	Now
RCU meets 100,000 reads/s at 8 readers	unchanged (98.25 M paced, 47.97 M unthrottled)
RCU scales to 8 readers; the lock stores stop at 4	unchanged in substance; the lock stores' numbers changed (§2)
"The shared mutex starves its writer at 8 readers"	**true only at one lock placement**: at offset 0 the paced writer gets 1,588 of 10,000 updates/s, at offset 48 all 10,000
"The global mutex reaches 21.7 M reads/s at 4 readers and stops scaling after 4"	**not reproduced**: 16.91 M (offset 48), 16.31 M (offset 0); it gains nothing from more readers. Neither the wrapper nor the placement accounts for the earlier figure (§7)
"The reclaim batch changes reclamation throughput and memory"	**withdrawn**: no effect of the batch on any measure (§3)
"Stop-the-world reads at one eleventh of the bounded rate"	**corrected**: about one seventh (7.18 against 48.30 M/s)
Bounded reclamation reduces the worst reader pause 28× / 87×	**42× / 116×**
Transactions, joins, copy-on-write costs	unchanged within a few percent (§4)

All three campaigns are kept: `results/part_d/README.md`; the cell-by-cell comparison is `results/part_d/summary/original_vs_correcte`
No ratio in this report mixes campaigns.

1. Environment and method

Machine. AMD Ryzen AI 7 350, 8 cores / 16 logical CPUs (cores 0, 2, 4, 6 reach 5.09 GHz; cores 1, 3, 5, 7 reach 3.51 GHz; CPU N+8 is the SMT sibling of N), L3 16 MiB, 30 GiB RAM, 8 GiB swap (unused). Ubuntu 26.04, Linux 7.0.0-28, GCC 15.2.0, Release build `-O3 -g -DNDEBUG -std=c++17 -fno-omit-frame-pointer`. Mains power, `performance` governor and EPP on all CPUs at each of 68 checkpoints (start, after every repetition, end); `kernel.perf_event_paranoid = 1`. Idle CPU sampled at every checkpoint: 98–99 %; no process other than the session's own terminal above 5 % CPU. Battery full; ACPI temperature 43–88 °C. Campaign from 2026-10-13T11:58:44 to 2026-10-13T12:59:05 (`conditions.log`, `env/`).

Timing track and profiling track.

- **Timing.** Each measured executable is started directly: no `perf`, no `/usr/bin/time`, no tracer (each run's file records its command and `# wrapper: none`; the aggregator and a verification gate reject a timing run started any other way). Throughput, latency, stalls, backlog and store memory come only from these runs.
- **Profiling**, after all timing stages, under `profiling/`: every track C, D.2 and D.3 cell again under `perf stat` with `/usr/bin/time` (2 repetitions: counters, context switches, peak resident set); `perf record` at 1,999 Hz with frame pointers for 14 flame graphs; `perf c2c` for 9 cases. Their throughput appears only in §5.

Lock placement. `concurrent_bench --lock-offset N` constructs the store and, if its lock object is not at offset N of a cache line, destroys it and constructs it again after a small allocation. Global-mutex and shared-mutex cells are measured at **offset 48** (the first 16 bytes of the lock end one line, the rest begins the next) and at **offset 0** (cache-line aligned; offsets 16 and 32 behaved like 0 in the diagnostic). Neither is "the" result and neither is better everywhere. Where a single lock-store number is compared with RCU, the higher of the two is used. The RCU store's readers take no lock.

Thread placement. Reader (or joiner) thread i is pinned to the i-th of CPUs 0, 2, 4, 6, 1, 3, 5, 7: one logical CPU per physical

core, fast cores first. The writer is pinned to CPU 15. The store's cleaner thread is not pinned (the store has no interface for it).

Two workload tracks, never in one table or figure.

- **Track R — supplied generator, preload-only.** The four drivers linked with the unmodified `supplied/workload_gen.cpp`, default parameters. Its writer loads all trades and *finishes before any reader starts*: there is no writer in these numbers. 20 % of its reads use `rng() % (m·n)` while trade ids are `i·100000 + j`, so only 100 of those 100,000 possible keys exist: measured hit rate **80.02 %**.
- **Track C — writer active.** `benchmarks/concurrent_bench.cpp`: the same schema, hot set and 80/20 mix, every requested key present, and one writer thread overwriting existing trades for the whole measured window (0.5 s warm-up, 5 s measured). Every read is validated; the writer's successful updates are counted. Two writer modes: paced at 10,000 updates/s, and unthrottled.

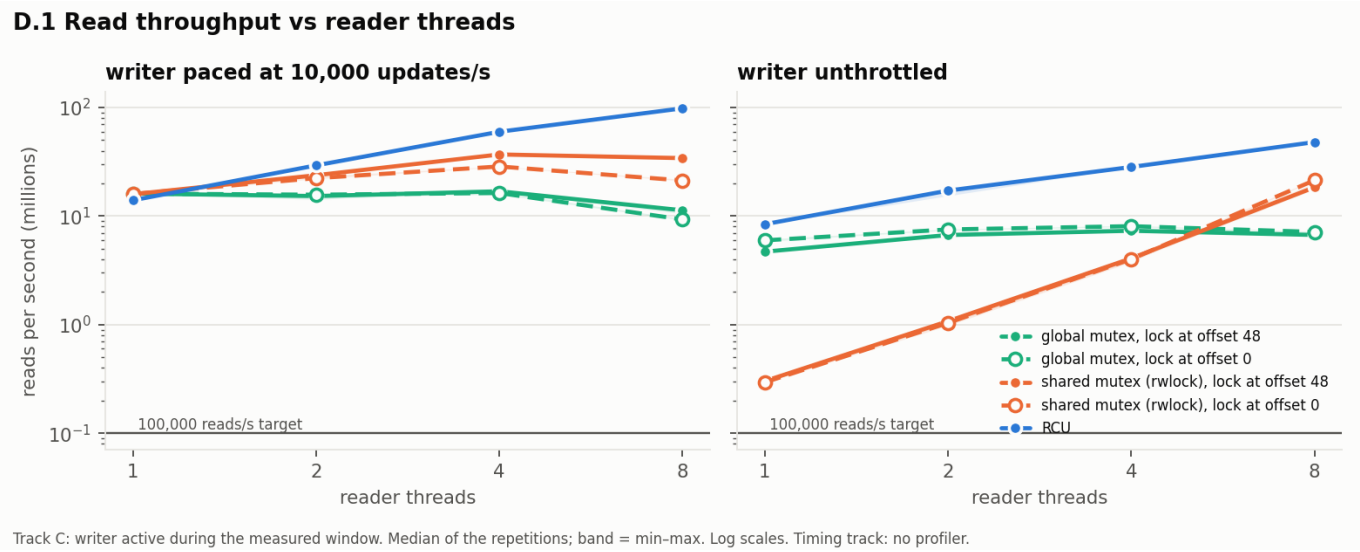
D.1 asks for the generator "with one background writer thread active throughout". The supplied generator cannot do that, so both tracks are reported and labelled.

Caveats that apply to all throughput numbers. Both harnesses time every `get` with two clock reads; in the profile of the RCU store with no writer those clock reads are 36 % of all cycles. The numbers are therefore lower bounds on what the stores can do, equally for all four.

Comparing RCU with the lock variants compares two whole designs. The Part A stores update one slot in place under a lock; the RCU store copies the 24 KiB segment it touches and a 2 KiB directory for every update and reclaims the old ones later. The difference between them is not the cost of a synchronization primitive alone.

2. D.1 — point-query throughput and the 100,000 RPS target

2.1 Track C: one writer active throughout



Track C: writer active during the measured window. Median of the repetitions; band = min–max. Log scales. Timing track: no profiler.

Reads per second (millions), median [min–max]; writer paced at 10,000 updates/s:

Readers	global mutex, lock at 48	global mutex, lock at 0	shared mutex, lock at 48	shared mutex, lock at 0	RCU
1	16.12 [16.00–16.22]	16.10 [15.83–16.26]	15.90 [15.87–15.97]	15.98 [15.90–16.17]	13.97 [13.92–14.02]
2	15.28 [14.26–16.13]	15.73 [15.07–15.89]	23.77 [23.63–24.11]	22.36 [22.05–23.28]	29.30 [29.10–29.57]
4	16.91 [16.75–17.11]	16.31 [15.84–16.61]	36.93 [36.64–37.35]	28.61 [28.37–28.68]	59.76 [59.59–59.91]
8	11.30 [10.53–11.51]	9.30 [9.20–9.44]	34.17 [34.11–34.45]	21.27 [21.10–21.76]	**98.25 [97.69–98.36]**

Writer unthrottled:

Readers	global mutex, lock at 48	global mutex, lock at 0	shared mutex, lock at 48	shared mutex, lock at 0	RCU
1	4.70 [4.66–4.92]	5.97 [5.86–6.12]	0.30 [0.30–0.30]	0.29 [0.29–0.30]	8.41 [8.35–8.50]
2	6.69 [6.57–6.82]	7.53 [7.43–7.59]	1.08 [1.02–1.12]	1.04 [0.98–1.08]	17.16 [15.38–17.50]
4	7.32 [7.19–7.37]	8.07 [8.00–8.19]	4.08 [3.88–4.28]	4.01 [3.78–4.08]	28.30 [28.17–28.38]
8	6.70 [6.64–6.76]	7.15 [7.06–7.22]	18.39 [17.64–20.01]	21.36 [21.30–21.77]	**47.97 [47.66–48.04]**

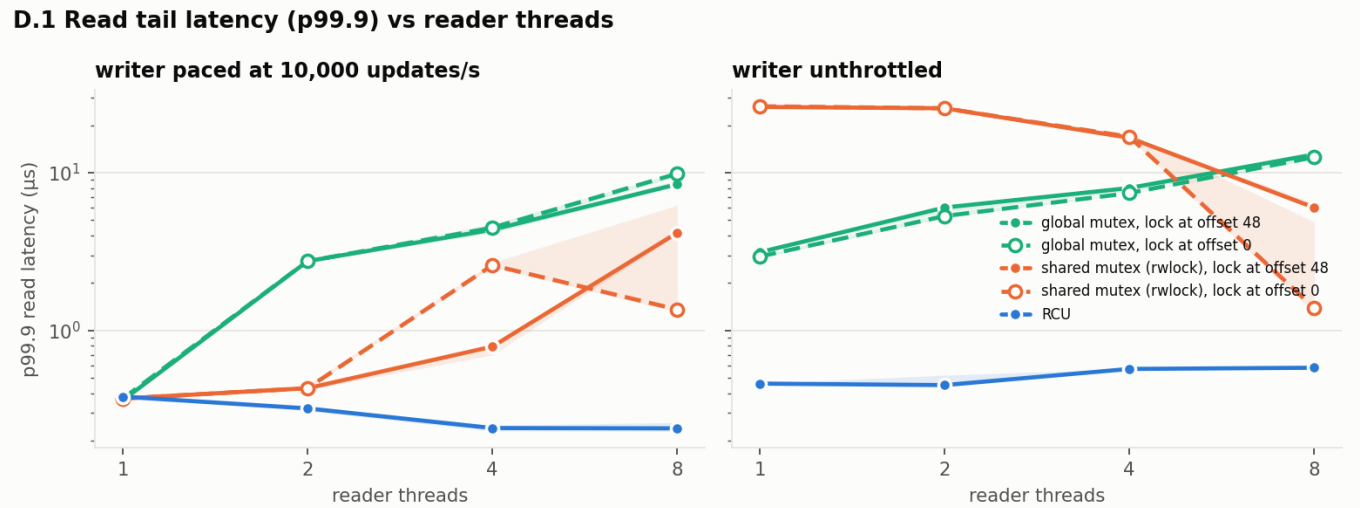
Every read in these tables found its key (hit rate 100 %; reads attempted = reads successful).

Read latency at 8 readers (µs):

Store, writer	p50	p99	p99.9	max, median run / worst run
global mutex at 48, paced	0.100	4.672	8.448	229 / 1,035
global mutex at 0, paced	0.210	5.376	9.856	221 / 311
shared mutex at 48, paced	0.140	0.641	4.160	669 / 2,749
shared mutex at 0, paced	0.391	0.732	1.360	3,008 / 3,969
RCU, paced	0.050	0.160	0.240	161 / 183
global mutex at 48, unthrottled	0.641	6.976	13.056	297 / 2,562
global mutex at 0, unthrottled	0.511	6.656	12.544	295 / 2,019

Store, writer	p50	p99	p99.9	max, median run / worst run
shared mutex at 48, unthrottled	0.130	4.544	6.016	1,947 / 3,756
shared mutex at 0, unthrottled	0.390	0.721	1.392	1,974 / 3,631
RCU, unthrottled	0.121	0.401	0.581	146 / 167

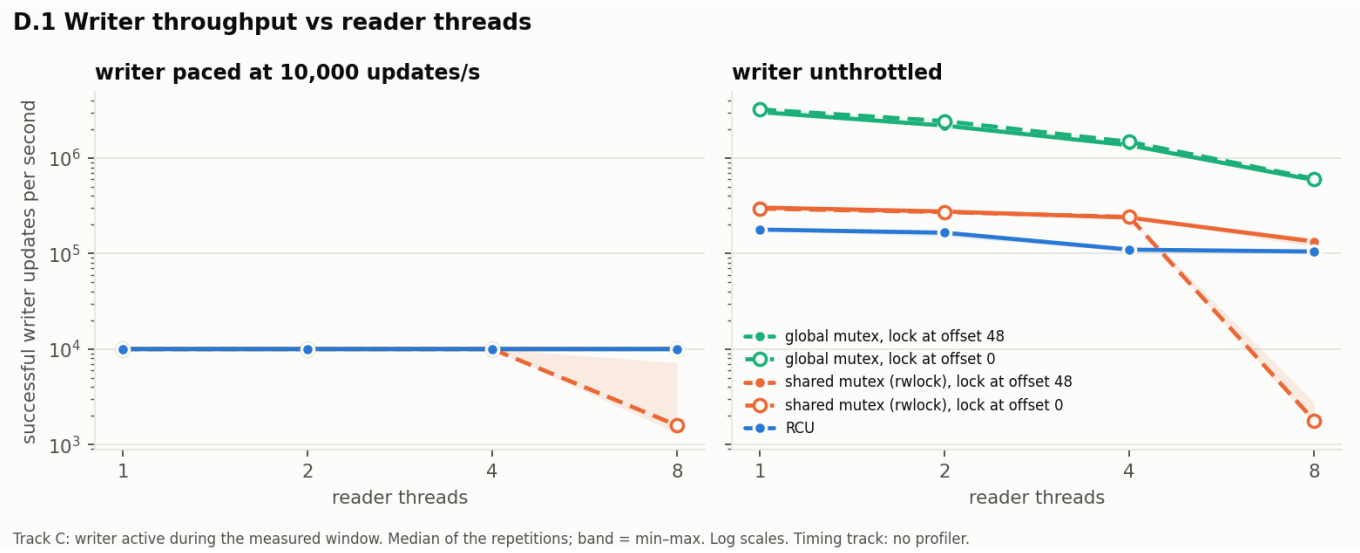
The maximum of a run is one sample in several hundred million. In this session it is about 140–160 µs in almost every cell, including one reader on an otherwise idle machine and the store with no writer at all, so that floor belongs to the machine, not to any store.



Successful writer updates per second:

Readers	global mutex at 48, unthrottled	global mutex at 0, unthrottled	shared mutex at 48, unthrottled	shared mutex at 0, unthrottled	RCU, unthrottled	shared mutex at 0, **paced** (target 10,000)
1	3,027,680 [2,953,080–3,101,340]	3,230,830 [3,179,570–3,320,920]	303,095 [300,877–305,473]	295,395 [294,552–299,210]	178,273 [175,266–179,942]	10,000
2	2,184,800 [2,176,410–2,247,110]	2,438,970 [2,401,870–2,454,590]	275,604 [269,407–281,490]	271,795 [266,300–277,894]	165,403 [151,269–171,393]	10,000
4	1,360,100 [1,348,990–1,389,110]	1,478,450 [1,454,820–1,484,880]	237,984	241,538 [238,035–242,912]	109,922 [106,051–110,556]	10,000
8	588,168 [586,100–598,874]	607,689 [603,562–617,354]	133,464 [117,662–146,792]	**1,787** **[1,646–2,766]**	104,956 [103,603–108,750]	**1,588** **[1,314–7,193]**

Every other paced cell delivers its 10,000 updates/s (medians 10,000–10,000).



One thread only: the unsynchronized store reads 18.32 M/s alone and 16.78 M/s while doing one update per 100 reads on the same thread; it is never run with concurrent threads. The RCU store with no writer reads 16.06 M/s on one thread and 112.79 M/s on eight.

With the generator's own key distribution *and* a paced writer (8 readers), lookups/s attempted and reads/s successful, hit rate 80.02 % in every case: global mutex 14.17 / 11.34 M (lock at 48) and 10.34 / 8.28 M (at 0); shared mutex 34.68 / 27.75 M and 22.74 / 18.20 M; RCU 104.24 / 83.41 M.

2.2 Where the lock-based stores stop scaling

Rule used: the largest reader count whose median is more than 5 % above the best median at any smaller count.

- **Global mutex: it does not scale.** With the paced writer the best count is 1 reader at offset 48 and 1 at offset 0: 16.12, 15.28, 16.91 M/s at 1, 2, 4 readers, then a **fall** to 11.30 M/s at 8 (9.30 M/s at offset 0). With the unthrottled writer it rises until 4 readers (4.70 → 7.32 M/s) only because more readers take the lock from the writer more often: the writer falls from 3,027,680 to 588,168 updates/s. Under `perf record` at 8 readers, 79 % of sampled on-CPU cycles are in the kernel (futex wait and wake), 11 % in the mutex functions and 1.3 % in the lookup (`profiling/flamegraphs/global_lock_r8_writer_max.svg`). p99 read latency grows from 0.16 µs at 1 reader to 4.7 µs at 8.
- **Shared mutex: after 4 readers with a paced writer, at both placements, but what happens at 8 differs.** At offset 48: 15.90 → 23.77 → 36.93 → 34.17 M/s: a plateau, and the writer keeps its 10,000 updates/s. At offset 0: 15.98 → 22.36 → 28.61 → 21.27 M/s: a fall, and **the writer is starved** (1,588 [1,314–7,193] of 10,000). Readers do not exclude each other, but every `get` writes the lock's reader count twice: at 8 readers 50 % (offset 48) and 59 % (offset 0) of on-CPU cycles are in `pthread_rwlock_rdlock/unlock`. With an unthrottled writer and few readers the opposite happens: the writer holds the lock and reads collapse to 0.30 M/s at 1 reader. The rise to 18.39 M/s (offset 48) or 21.36 M/s (offset 0) at 8 readers is the readers taking the lock away from the writer, not parallel speed-up: at offset 0 the writer is left with 1,787 updates/s, at offset 48 with 133,464.
- **RCU: still rising at 8 readers** in both writer modes: 13.97 → 29.30 → 59.76 → 98.25 M/s paced (7.0× from 1 to 8 readers; the last four readers run on the slower cores and one shares its core with the writer), 8.41 → 17.16 → 28.30 → 47.97 M/s unthrottled. At 8 readers with the paced writer that is 2.9× the better shared-mutex placement and 8.7× the better global-mutex placement.

The lock's position in its cache line. Same binary, same arguments, 8 readers, paced writer:

	lock at offset 48	lock at offset 0
shared mutex, reads/s (M)	34.17	21.27
shared mutex, writer updates/s	10,000	1,588
shared mutex, sampled loads that hit a line modified by another core (`perf c2c`)	11.1 %	41.0 %
shared mutex, share of those in `pthread_rwlock_unlock`, on one line	93 %	96 %
global mutex, reads/s (M)	11.30	9.30

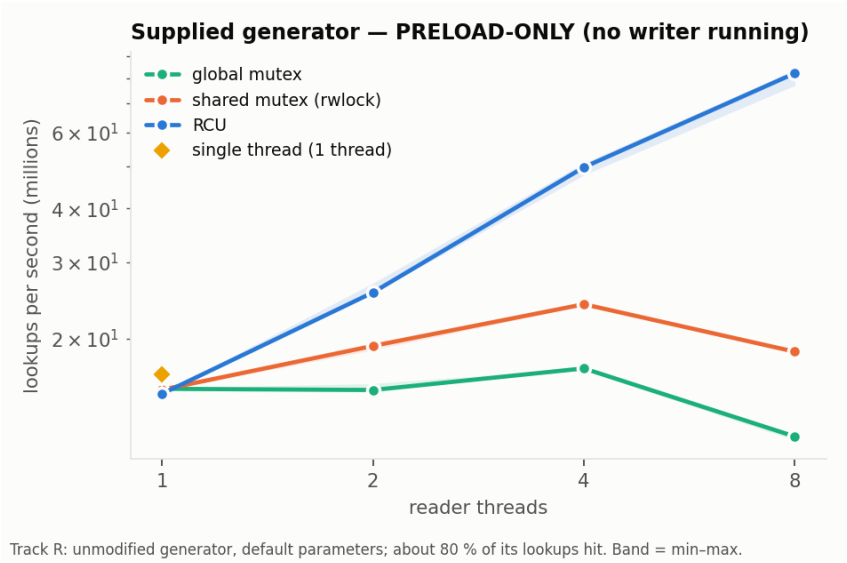
At offset 48 only the lock's first 16 bytes (the words every lock and unlock modifies) are in one cache line and the rest of the lock is in the next. In glibc 2.43 (field offsets checked on this machine, `results/part_d_diagnostics/glibc_lock_layout.txt`; the order of accesses is from the glibc sources) `pthread_rwlock_unlock` first reads `__cur_writer` (byte 24) and `pthread_mutex_lock` first reads `__kind` (byte 16) before the atomic update of the first word. With both on one line the read fetches the contended line and the update must then acquire it again; apart, the read is served from a line nobody writes. The `perf c2c` rows above are consistent with that, and so are all eight placements tried, but **this explanation is a hypothesis: no experiment here isolates it.** The effect itself is measured. It is not uniformly in favour of offset 48: with the unthrottled writer the global mutex is *slower* at 48 (4.70 against 5.97 M/s at 1 reader).

2.3 The 100,000 reads/s target

RCU store, 8 readers	Median reads/s	Minimum of 5 runs	100,000/s
Track C, writer paced at 10,000/s	98,250,000	97,690,500	**met**
Track C, writer unthrottled (104,956 updates/s)	47,970,200	47,658,400	**met**
Track R, supplied generator (preload-only)	82,169,200 lookups/s	77,020,600	**met**

The target is met by a factor of about 477 to 982 in every configuration measured, with p99.9 read latency of 0.24 µs (paced writer) and 0.58 µs (unthrottled). These are unwrapped runs. Because the target is met, the assignment's fallback ("report the number you achieved and explain the remaining bottleneck, backed by a flame graph") does not apply; the flame graphs were taken anyway and are used in §2.2 and §2.5.

2.4 Track R — supplied generator, preload-only



Track R: unmodified generator, default parameters; about 80 % of its lookups hit. Band = min–max.

Readers	single thread	global mutex	shared mutex	RCU
1	16.54 [16.47–16.63]	15.30 [15.26–15.37]	15.23 [15.16–15.27]	14.90 [14.84–14.96]
2	—	15.20 [15.11–15.73]	19.23 [18.82–19.24]	25.56 [25.45–26.84]
4	—	17.06 [16.91–17.07]	24.01 [23.86–24.36]	49.79 [47.79–50.92]
8	—	11.87 [11.64–11.96]	18.65 [18.54–18.83]	82.17 [77.02–83.53]

Lookups per second (millions), **no writer running**, 80.02 % of lookups hit. The lock placement of the drivers cannot be chosen (their arguments are fixed and the supplied file is not edited); each driver reports the offset it got: **0 for the shared-mutex driver, 48 for the global-mutex driver.** The shared-mutex driver's fall from 4 to 8 readers is therefore the offset-0 behaviour of §2.2, and no offset-48 figure exists for it. The shape is otherwise that of track C: the mutex stores do not gain beyond 4 readers, RCU keeps rising. p99.9 at 8 readers as printed by the generator: 7.8 µs (global mutex), 1.2 µs (shared mutex), 0.2 µs (RCU).

2.5 What RCU costs, and what limits it

Memory and reclamation. Every update copies 25,974 bytes (one 24 KiB segment and the 2 KiB directory) for a 32-byte value. With the paced writer the store holds 11 MiB at the end of every run and the backlog is empty. **With the unthrottled writer the cleaner does not keep up beyond 2 readers:**

Readers (unthrottled writer)	Writer updates/s	Memory held at end, MiB (median; max)	Retired versions not yet reclaimed at end	Cleaner CPU, s of 5.5
1	178,273	31; 32	0	4.52
2	165,403	31; 34	0	4.88
4	109,922	734; 781	26,856	5.45
8	104,956	3,945; 4,608	148,852	5.30

The cleaner thread is busy for 4.5–5.4 of the 5.5 s at every reader count. At 8 readers the backlog at the end of a 5 s window is 148,852 versions and 3.9 GiB, about 27,064 versions and 0.70 GiB per second, for as long as the writer runs flat out. The store has no back-pressure (a deliberate choice, decision D-040), so this ends only when the writer slows down or memory runs out. It drains completely afterwards (validated in every run). The audited Part B binary behaved the same when run alternately with the first campaign's binary (a diagnostic outside the campaigns, `results/part_d_diagnostics/part_b_reference_backlog.txt`), so it is not something Part C introduced. **A sustained unthrottled writer with 4 or 8 readers is not a steady state for this store on this machine.** The 10,000 updates/s workload is.

Reads slow down by half next to an unthrottled writer (98.25 → 47.97 M/s at 8 readers; 16.06 → 8.41 M/s for a single reader with idle cores all around it). What the profiling track says (counters are per process and come from profiled runs; RCU cells move by at most a few percent under the profilers, §5):

- It is not lock or scheduler contention: readers take no lock, and the slowdown is uniform across reader threads (slowest/fastest 4.80/7.22 M/s against 11.49/16.48 M/s with no writer) and present with a single reader.
- Instructions per read hardly change (447 → 547); cycles per read go from 337 to 909 and instructions per cycle from 1.33 to 0.60. The readers are stalled, not doing more work.
- `perf c2c` (AMD IBS load sampling): of the loads with a known data source, those served from L3 rise from 1.3 % to 3.5 % and those that hit a line modified by another core from 0.5 % to 2.8 %.
- **The hypothesis that writer-updated segment owner counts share cache lines with reader data (audit item R8) is only weakly supported.** Of 797 such "modified elsewhere" load samples at 8 readers, 57 % were taken by the writer's own commit and 20 % by the functions taking and releasing references (cleaner and writer): the writer and the cleaner contending with each other on owner and reference counts. **10.0 % (80 samples, on 69 different cache lines) were taken by the reader's lookup**, which is 0.10 % of all sampled loads. Readers do touch lines the writer or cleaner has just modified, but too rarely for that to explain a factor of two. The hypothesis is not established.
- What fits all of the above, without being proven by it: copy-on-write *relocates* what readers read. Each update publishes a new 24 KiB segment at a new address, so the copy of that partition a reader has in its own caches is useless and the next probe has to fetch lines from L3 or from the writer's core; at 104,956 updates/s that is 2.7 GB/s of freshly written index flowing through a 16 MiB shared L3.
- The profile cannot separate this further. In the flame graphs the reader threads' cycles shift from the lookup (58 % → 17

%) to the harness's clock reads (36 % → 50 %); a timestamp read waits for the loads before it, so stall time is charged to it. That shift is an artefact of attribution, not evidence that reading the clock became slower.

Where the writer's and the cleaner's time goes (unthrottled writer, share of sampled on-CPU cycles of the whole process): with 1 reader the cleaner takes 32 % and the writer's `put` 23 %; with 8 readers 9 % and 6 % (the readers' share grows; the cleaner still uses a whole core). The cleaner costs more cycles than the writer whose versions it destroys, which is why it is the one that falls behind.

No optimization was attempted. A test that would decide the read slowdown: per-thread hardware counters for the reader threads alone, and the same workload against a store that updates in place.

3. D.2 — reclamation-batch tuning

`set_reclaim_batch(n)`: one reclamation pass frees at most n retired versions. Settings 1, 8, 64, 256, 1024; reclamation disabled; and the naive stop-the-world sweep (all readers frozen by a lock, the whole retire list freed at once, as soon as it holds 1,024 versions). RCU store, 8 readers, unthrottled writer, 3 s windows (1 s with reclamation disabled, which retains everything; guarded by a 16 GB limit that was never reached). In every epoch-based run the largest number of versions freed in one pass equalled the batch (1, 8, 64, 256, 1,024) and never exceeded it, in both workloads.

Two workloads, **not comparable with each other**:

- **A, steady point gets:** the D.1 benchmark (100,000 trades of 32 bytes).
- **B, backlog-building:** `benchmarks/reclaim_bench.cpp`, 2,000 keys of 4 KiB, and a thread that holds a Snapshot for 20 ms out of every 50 ms. While it is held nothing retired after it can be freed, so thousands of versions accumulate and are released in passes. At the end the writer stops, the last snapshot is released, and the time to drain is measured with the readers still running.

3.1 Results

Workload A, what a reader sees:

Setting	Reads/s (M), median [min–max]	p50 µs	p99 µs	p99.9 µs	Longest single reader stall, µs: median run / worst run
batch 1	48.58 [48.51–48.74]	0.120	0.401	0.571	142 / 453
batch 8	48.62 [48.40–48.73]	0.121	0.401	0.571	149 / 501
batch 64	48.30 [47.96–48.45]	0.121	0.401	0.572	146 / 241
batch 256	48.45 [48.27–48.65]	0.121	0.401	0.571	164 / 245
batch 1024	48.46 [48.23–48.62]	0.121	0.401	0.571	141 / 147
disabled (1 s)	73.83 [73.59–74.35]	0.070	0.291	0.451	140 / 147
stop-the-world	7.18 [6.55–7.27]	0.301	1.002	1.808	**6,819 / 8,799**

Workload A, writer and cleaner:

Setting	Writer updates/s	Passes	Most versions freed in one pass	Versions reclaimed/s	Backlog max	Memory at end, MiB	Cleaner CPU s
batch 1	108,776	315,827	1	105,268	79,240	2,097.8	3.36
batch 8	108,180	38,419	8	102,145	83,794	2,217.1	3.36
batch 64	110,122	5,118	64	107,548	77,330	2,048.7	3.37
batch 256	107,915	1,519	256	102,072	84,576	2,238.1	3.38
batch 1024	108,887	469	1,024	103,554	81,997	2,171.2	3.37
disabled (1 s)	83,815	—	—	0	225,994	4,453.0	0.00
stop-the-world	158,638	—	—	188,914	1,404	43.4	2.22

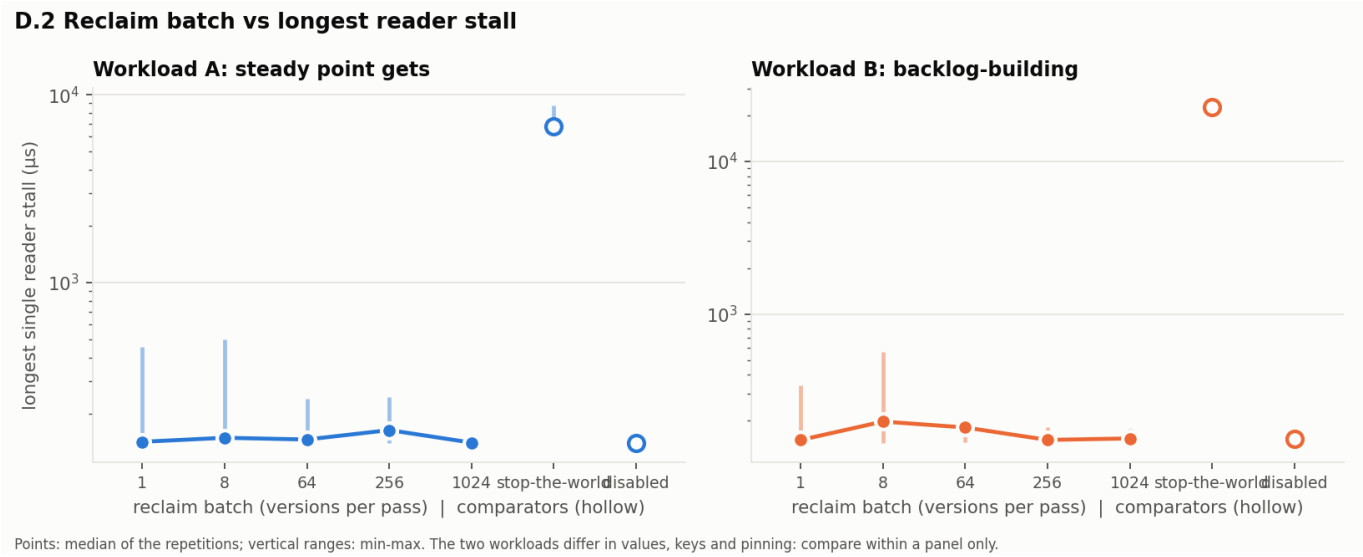
Workload B, what a reader sees:

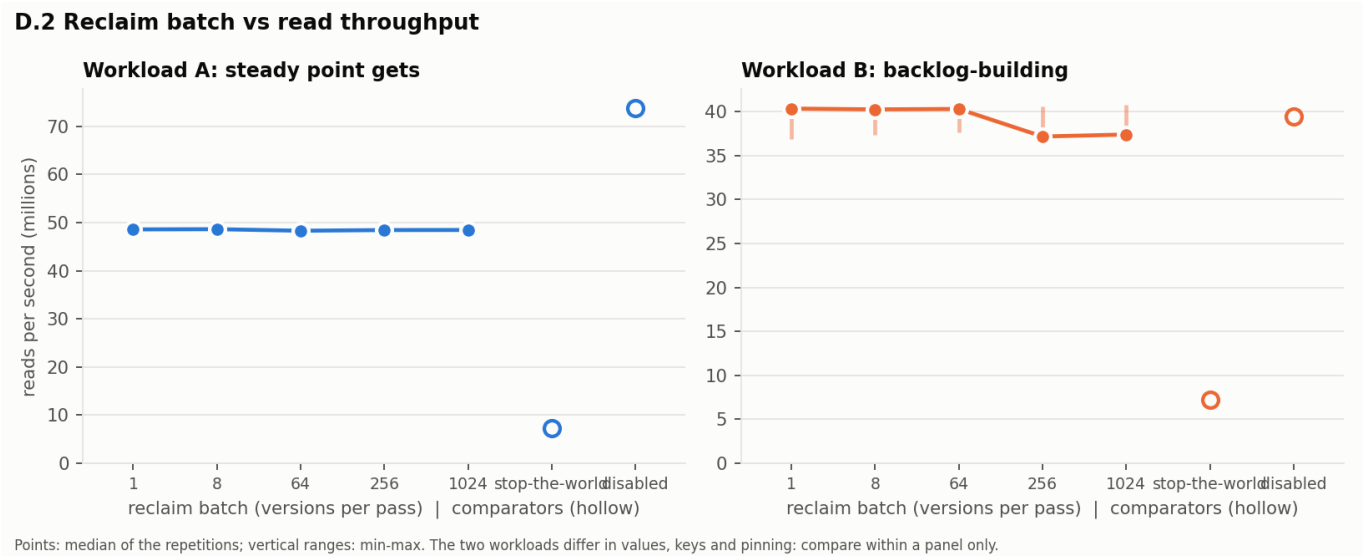
Setting	Reads/s (M), median [min–max]	p50 µs	p99 µs	p99.9 µs	Longest single reader stall, µs: median run / worst run
batch 1	40.36 [36.90–40.56]	0.161	0.320	0.491	149 / 341
batch 8	40.27 [37.34–40.85]	0.161	0.320	0.491	197 / 563
batch 64	40.32 [37.62–40.51]	0.161	0.320	0.491	181 / 201
batch 256	37.18 [37.10–40.62]	0.161	0.311	0.491	150 / 182
batch 1024	37.41 [37.09–40.80]	0.161	0.321	0.491	153 / 177
disabled (1 s)	39.44 [39.35–39.58]	0.151	0.291	0.471	152 / 153
stop-the-world	7.23 [6.40–7.41]	0.290	0.751	5.696	**22,814 / 23,380**

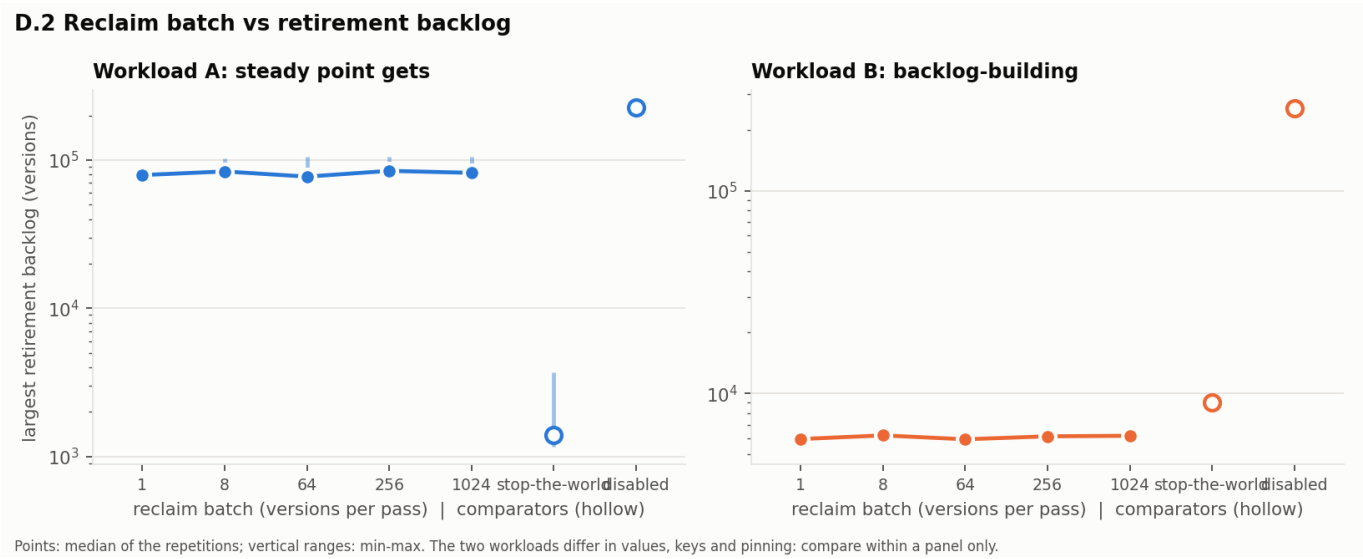
Workload B, writer and cleaner:

Setting	Writer updates/s	Passes	Most versions freed in one pass	Versions reclaimed/s	Backlog max	Memory max, MiB	Cleaner CPU s	Final cleanup: backlog → ms to drain
batch 1	222,705	776,136	1	258,239	5,965	67.7	2.77	6,002 → 16.1
batch 8	222,679	99,496	8	258,988	6,215	68.9	2.52	6,228 → 19.2
batch 64	223,468	14,735	64	259,632	5,948	67.8	2.67	5,957 → 18.9
batch 256	223,382	5,717	256	259,591	6,151	68.8	2.58	6,179 → 14.9
batch 1024	225,531	3,051	1,024	260,861	6,189	68.2	2.72	6,213 → 20.1
disabled (1 s)	171,362	—	—	0	255,440	1,761.8	0.00	—
stop-the-world	293,161	—	—	341,470	9,014	89.2	2.04	8,444 → 12.8

The two comparators do not make bounded passes, so those columns are empty for them.







3.2 What the batch size trades off here: nothing that was measured

The assignment expects "a bigger batch should mean higher throughput but a longer worst-case stall". **That is not what was measured.** Across a thousandfold range of batch sizes, in both workloads, no measure taken depends on the batch:

- **Read throughput.** 48.30–48.62 M/s in workload A, with p50, p99 and p99.9 equal to within 1 ns. In workload B the medians are 37.18–40.36 M/s, but every batch size has runs at both about 37 and about 40.5 M/s (ranges in the table): the run-to-run spread, not the batch, sets the median.
- **The longest reader stall.** Medians of 142, 149, 146, 164, 141 µs for batches 1 to 1024 in workload A and 149, 197, 181, 150, 153 µs in workload B: no trend. With reclamation **disabled** the longest stall is 140 µs (A) and 152 µs (B), the same; it is the machine's floor seen in §2.1. There is no evidence that a reclamation pass stalls a reader, at any batch size. The cleaner works on its own thread and takes no lock a reader takes.

- **Reclamation throughput, backlog and memory.** 102,072–107,548 versions/s in workload A and 258,239–260,861 in workload B, not ordered by batch; the largest backlog is 5,948–6,215 versions in B for every batch; the cleaner uses the same CPU time (3.36–3.38 s in A). Draining the final backlog of workload B takes 16.1, 19.2, 18.9, 14.9, 20.1 ms for batches 1 to 1024: no trend. What does change is the number of passes (315,827 at batch 1, 469 at batch 1024), each of them correspondingly longer.

The first report said that a batch of 1 reclaims more slowly and holds more memory. That was measured inside `perf stat` and is **not reproduced** by the unwrapped runs; it is withdrawn.

So for this design, on this machine, the tunable is a bound on the cleaner's work between two checks of the epoch and nothing more: it is what makes a pass finite and its work accounted for, and it has no measurable price or benefit between 1 and 1024. The design reason is that the cost the assignment expects a batch to amortize, stopping or signalling readers, does not exist here: readers are never stopped.

3.3 The two reference points

- **Reclamation disabled** reads fastest in workload A (73.83 against about 48.30 M/s) because nothing is being freed: no cleaner competing for a core and for the allocator. It retained 4.3 GiB in 1.5 s in workload A and 1.7 GiB in workload B. It is not a usable configuration. (In workload B it reads at 39.44 M/s, inside the range of the batch runs.)
- **Stop-the-world** frees memory fastest (188,914 versions/s; 43 MiB held where the epoch-based runs hold about 2 GiB) and lets the writer run faster (158,638 against about 110,122 updates/s), because the sweep has no batch limit and nothing else runs meanwhile. It pays with the readers: every `get` takes a shared lock, so **read throughput falls to 7.18 M/s (one 7th) and the longest reader stall rises to 6.8 ms in workload A and 22.8 ms in workload B.** Its lock is inside the engine and was not placed; in a diagnostic its read throughput varied between 5.6 and 7.4 M/s with the heap layout, so that figure is good to about ±15 %. The stall is the length of a sweep and did not vary that way.

Bounded reclamation against stop-the-world, worst reader pause (median over 5 runs of each run's longest stall; the epoch figure is the batch size with the largest median):

Workload	Stop-the-world	Epoch-based (worst batch)	Reduction
A, steady	6,819 µs	164 µs (batch 256)	42×
B, backlog-building	22,814 µs	197 µs (batch 8)	116×

Since the epoch-based maximum is the machine's floor rather than anything reclamation does, these ratios are lower bounds on the reduction and would be larger on a quieter machine.

4. D.3 — join and multi-key writes

4.1 Index-nested-loop join, writer active and paused

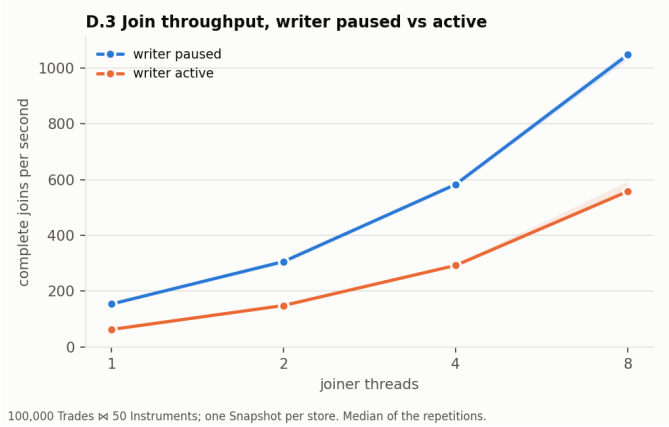
The join of Part C.2 (`benchmarks/join.h`): for every key that `keys()` returns for the Trades table, fetch the trade, probe the Instruments table's secondary index with its symbol, and fetch every matching instrument. 100,000 trades, 50 instruments, 100,000 joined rows per join; every join is validated (complete; pairs match; the inner table of one version).

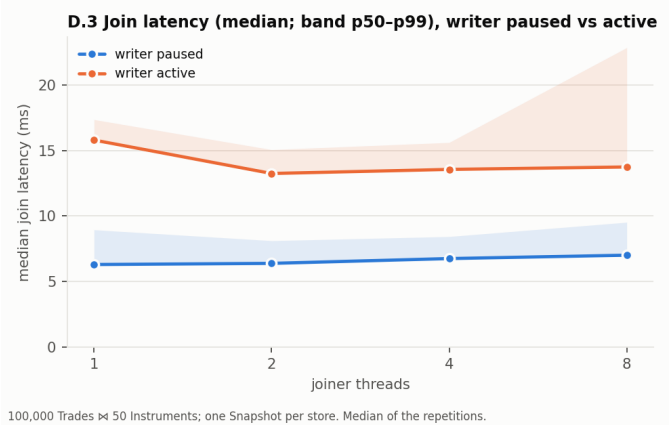
Snapshots: one per store. The join takes one Snapshot of Trades and one of Instruments at its start and uses each for every call on its own store. Each table is read from one immutable version for the whole join; the two versions are stable per-store versions and are not guaranteed to be one atomic instant; and it is two handles, not the "exactly one Snapshot handle" of the assignment's wording (a recorded deviation, decision D-060).

When active, the writer issues single Trade puts that change the trade's symbol and, after every 64 of them, one transaction rewriting all 50 instruments.

Joiners	Writer	Joins/s	Joined rows/s (M)	p50 ms	p99 ms	p99.9 ms	max ms, median run / worst run	Memory max, MiB	Backlog max
1	paused	154.2 [152.0–155.4]	0.00	6.29	8.94	10.90	10.90 / 11.61	9.1	0
1	active	63.0 [62.2–63.8]	0.00	15.79	17.35	17.67	17.67 / 18.25	161.6	5,909
2	paused	306.5 [299.3–307.8]	0.00	6.38	8.11	10.51	10.51 / 11.07	9.1	0
2	active	148.9 [148.4–150.2]	0.00	13.24	15.06	15.30	15.30 / 15.66	138.8	5,025
4	paused	583.0 [576.4–591.6]	0.00	6.75	8.42	10.46	10.49 / 10.89	9.1	0
4	active	292.5 [288.9–297.8]	0.00	13.55	15.60	15.75	15.75 / 15.93	139.1	5,001
8	**paused**	**1,048 [1,032–1,062]**	0.00	**7.01**	9.51	11.71	12.21 / 12.67	9.1	0
8	**active**	**557.7 [552.9–593.3]**	0.00	**13.74**	22.84	25.19	25.55 / 25.97	307.5	11,546

Writer while the joins ran: 173,176 Trade puts/s and 2,706 Instrument transactions/s with one joiner; 125,527 and 1,961 with eight.





What the concurrent writer costs a join. At 8 joiners: 1.96× the median latency (7.01 → 13.74 ms), 2.4× the p99 (9.51 → 22.84 ms) and 47 % of the throughput (1,048 → 558 joins/s). The same factor of about two as for point reads next to an unthrottled writer (§2.5), and plausibly for the same reason: the join never waits for the writer, but the rows and index segments it reads keep being republished at new addresses. A join is never blocked and never returns a wrong or partial result in either mode.

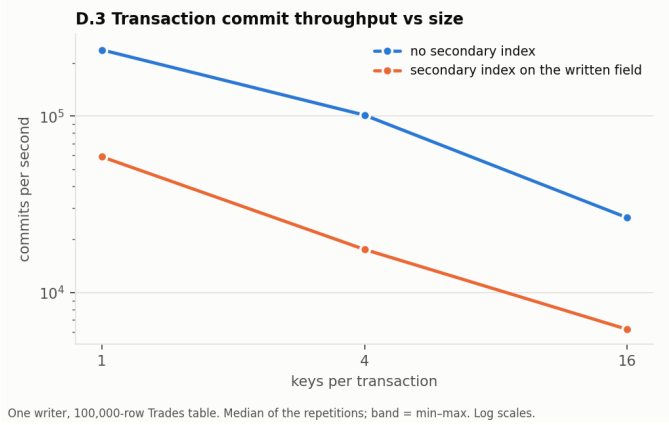
Joins hold reclamation back. A join pins both stores for its 7–26 ms, so everything retired meanwhile waits: up to 307 MiB held and 11,546 unreclaimed versions with 8 joiners and the writer active, against 9.1 MiB with the writer paused. All of it is reclaimed when the joins end.

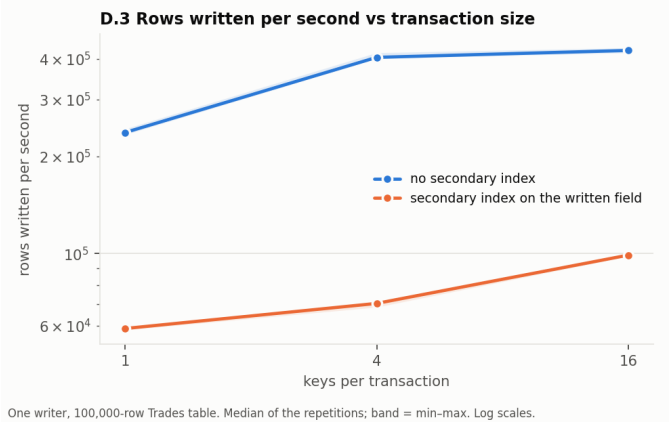
The generator's `use_join` path is not this join. It follows each successful `get` with one `index_get` on the Instruments store and discards the result: an index probe. On the RCU store, 8 readers, preload-only: 62.01 M operations/s against 81.53 M without the probe. Its probe passes an eighth symbol byte that the generator never writes; in these binaries that byte was 0 in all 100,000 stored trades and the probe matched **100 %** (measured from the store after each run).

4.2 Transaction commit throughput by size

One writer committing transactions of N Trades into the 100,000-row table, keys spread over the table (so N keys fall into N different hash partitions almost always). "No index" is the Trades table as the assignment loads it; "indexed" adds a secondary index on the field every write changes, the worst case for index maintenance.

Keys	Index	Commits/s	Rows/s	p50 µs	p99 µs	p99.9 µs	max µs, median run / worst run	Segments cloned per commit	Bytes copied per commit	Bytes copied per row
1	none	236,846 [233,312–243,026]	236,846	3.6	8.4	10.2	73 / 170	1.00	26,103	26,103
4	none	101,070 [100,194–104,454]	404,281	8.9	17.5	22.2	100 / 245	3.98	97,728	24,432
16	none	26,544 [26,115–27,025]	424,698	27.4	72.6	85.0	141 / 202	15.55	376,024	23,502
1	indexed	58,726 [58,141–59,035]	58,726	16.4	29.4	34.1	65 / 96	1.00	140,817	140,817
4	indexed	17,553 [17,094–17,662]	70,210	57.1	75.7	85.4	135 / 227	3.98	523,721	130,930
16	indexed	6,188 [6,141–6,206]	99,004	160.0	208.7	231.9	295 / 320	15.55	1,472,700	92,044





- **Commit throughput falls almost in proportion to the transaction size**: 236,846 → 101,070 → 26,544 commits/s for 1, 4, 16 keys, because each key lands in a different partition and each touched partition is one 24 KiB segment to copy. A commit costs about 24 KiB of copying per key plus a fixed 2 KiB directory and about 250 owner-count updates.
- **Rows per second rise** (236,846 → 404,281 → 424,698) because that fixed part and the single publication are shared by more rows; the gain is small since the per-key copy dominates.

- Each commit is still one publication, whatever its size. With a single writer and no reader the cleaner keeps up: the store never holds more than 16.7 MiB or 91 unreclaimed versions.
- **A secondary index on the written field costs about a factor of 4** (58,726 against 236,846 commits/s for one key): changing a row's indexed value rebuilds two of the 64 index partitions, about 115 KiB. With 100,000 rows and only 50 distinct values those partitions hold whole posting lists of 2,000 keys. This is the cost of copying index segments; the table the assignment indexes (Instruments, 50 rows) pays almost nothing, and the writer rewrote it in whole-table transactions 2,706 times per second during the join runs.

4.3 Where copy-on-write helps little, or hurts

Measured: the same Trade rows written as one transaction and as single puts.

Case	Publications	Elapsed ms	Rows/s	Segments cloned	Bytes copied	Bytes copied per row (payload 32)
100,000 rows, one transaction	1	13.26 [12.85–13.48]	7,541,510	256	6.1 MB	61
100,000 rows, single puts	100,000	555.37 [554.21–555.85]	180,061	100,000	2,610 MB	26,103
4,096 rows over all partitions, one transaction	1	0.77 [0.72–0.92]	5,347,100	256	6.1 MB	1,500
4,096 rows, single puts	4,096	23.91 [23.53–24.07]	171,297	4,096	107 MB	26,069
16 rows, one transaction	1	0.03	557,997	15	0.35 MB	21,632
16 rows, single puts	16	0.06	271,228	16	0.40 MB	25,088

Copy-on-write copies a whole 24 KiB segment to change one 32-byte row: **816 bytes copied per byte of payload** for a single-key update. That is where it hurts:

1. **Small, frequent, scattered writes.** Every update pays the full segment; that is why the global-mutex store, which updates one slot in place, completes 3.23 million updates/s where the RCU store completes 178,273 (D.1, 1 reader, unthrottled), and why the cleaner cannot keep up at 4 or 8 readers (§2.5).
2. **A very large transaction that touches most of the table**, the assignment's example. It is efficient *per row* (61 bytes per row for 100,000 rows: the segments are copied once and filled completely), but: it copies the entire index (6.1 MB here, the whole table); the old and the new version coexist, so memory doubles until the old one is reclaimed, and stays doubled for as long as any reader or snapshot is in the old version; and it is one 13 ms-long commit during which no other write can be published. For a table larger than the caches this is a full rewrite to make a change that an in-place store would make row by row.
3. **A medium transaction spread thinly** is the worst per row among the batched cases: 4,096 rows over all 256 partitions copy the same 6.1 MB as 100,000 rows (1,500 bytes per row), because every partition is touched and each is copied whole whatever fraction of it changes.
4. **Workloads with long readers.** A join or any long-held snapshot keeps every version published during it alive (307 MiB during 14 ms joins here); a lock-based store has no such cost because it has no old versions.
5. **Indexes on low-cardinality fields of large tables** (§4.2).

Where it helps: reads never wait (p99.9 of 0.24–0.58 µs against 1.4–13.1 µs for the lock stores at 8 readers), read throughput scales with cores, the writer cannot be starved by readers whatever the memory layout, and multi-row consistency for a reader costs nothing extra.

5. Profiling track: what the profilers do to the numbers

All figures in this section come from the profiled runs under `profiling/`; none of them is used anywhere else as a throughput or latency result. Full table for all 154 profiled runs: `summary/summary.md`, `summary/profiling_overhead.csv`.

Under `perf stat` (with `/usr/bin/time`), median of 2 runs against the timing track's median of 5, same cell and same lock placement:

Cell	Profiled / timing
RCU, 8 readers, paced writer	0.99
RCU, 8 readers, unthrottled writer	1.04
RCU, 8 readers, no writer	0.99
global mutex at 48, 8 readers, paced writer	1.25
global mutex at 0, 8 readers, paced writer	1.22
global mutex at 48, 2 readers, paced writer	0.86
global mutex at 48, 8 readers, unthrottled writer	1.18
shared mutex at 48, 8 readers, paced writer	0.93
shared mutex at 0, 8 readers, paced writer	1.00
shared mutex at 48, 1 / 2 / 4 readers, unthrottled writer	0.48 / 0.57 / 0.51
stop-the-world, workload A / workload B (reads/s)	0.71 / 0.41
batch 64, workload A / workload B (reads/s)	1.03 / 1.00
transactions of 1 / 4 / 16 keys, no index (commits/s)	0.98 / 1.02 / 0.99
join, 8 joiners, writer paused / active (joins/s)	0.99 / 1.10

`perf stat` in counting mode saves and restores its counters at every context switch. The cells it changes are the ones whose threads block in the kernel millions of times per run (global mutex: 2.5 million context switches in a 5.5 s run at 8 readers; RCU: 68,066), and the effect has either sign: slowing every handover can make a lock convoy shorter. RCU, transactions and joins move by a few percent at most.

`perf record` (1,999 Hz, frame pointers) and `perf c2c record` change the cells they observed by 0–5 %, with two exceptions, the RCU store at 2 readers (0.90) and the shared mutex at 8 readers with an unthrottled writer (1.16–1.56): that cell is the least stable in the timing track too (§6).

perf stat, 8 readers, unthrottled writer, whole process:

Store	CPUs utilized	Instructions per cycle	Context switches per second	Peak resident set, MiB
global mutex at 48	7.16	0.68	734,845	39
global mutex at 0	7.24	0.65	692,609	39
shared mutex at 48	5.64	0.63	530,564	23
shared mutex at 0	7.81	0.29	30,075	14
RCU	8.65	0.60	25,706	5,484

(The RCU process also runs its cleaner; its resident set is the unreclaimed backlog of §2.5.)

Stop-the-world, flame graph (`rcu_r8_stw`): 25 % of cycles in the shared-lock functions that every `get` must call in that mode, 19 % in the writer, 18 % in the sweeping cleaner and 9 % in the lookup.

6. Failed, corrected or inconclusive

- **"A bigger batch gives higher read throughput and a longer worst-case stall."** Not observed (§3.2), in two workloads and two independent campaigns of the timing kind.
- **"A batch of 1 reclaims more slowly and holds more memory"** (first report). Not reproduced without the profiler wrapper; withdrawn.
- **"The shared mutex starves its writer at 8 readers"** (first report). True at one lock placement, false at the other (§2.2).
- **"Owner counts share cache lines with reader data and cause the read slowdown next to a writer" (R8).** Only weakly supported: reader loads that hit a remotely modified line exist but are 0.10 % of sampled loads (§2.5). The slowdown is real and its cause is not established beyond "the readers are stalled on memory"; no change was made to the store.
- **Why the lock's cache-line offset matters** is a hypothesis consistent with the data, not a demonstrated mechanism (§2.2).
- **"The steady workload is steady."** It is not at 4 or 8 readers with an unthrottled writer (§2.5): the backlog grows throughout.

7. Limitations and threats to validity

- One machine, one compiler, one C library, a laptop with two kinds of cores. The lock placement effect in particular is a property of this glibc's lock layouts on this CPU; the two placements bracket it here and may not elsewhere.
- **The lock-store numbers are two points, not one.** A program that does not control where its lock lands gets one or the other by accident, and can change sides when an unrelated string changes length. Part A's accepted results were measured that way and are left as they are.
- **Three sessions of one machine differ by a few percent** even where nothing was wrapped or moved: the supplied generator's figures in this campaign are 0.93–1.01 of the first campaign's (`summary/original_vs_corrected.md`). Differences of that size between campaigns mean nothing.
- **Cells that remain uncertain.** (e) Global mutex at 2 and 4 readers with the paced writer: the first campaign gave 18.47 and 21.73 M/s (lock at offset 48, wrapped); this one gives 15.28 and 16.91 M/s at the same offset, and under the same wrapper 0.86× and 1.03× of that. So the earlier, higher figures are explained neither by the wrapper nor by the placement, and were not reproduced in either later session; a lightly contended mutex is evidently sensitive to something else that differs between sessions. The statement that survives all three campaigns is the weaker one: the global mutex never exceeds about 22 M reads/s and falls at 8 readers.
  (a) Shared mutex, 8 readers, unthrottled writer, lock at offset 48: reads 17.64–20.01 M/s and writer 117,662–146,792 updates/s across 5 runs, and 16–56 % higher reads under the sampling profilers: readers and writer share the lock in a ratio that is not stable. (b) Stop-the-world read throughput: about ±15 % with heap layout (its stall is not affected). (c) Workload B read throughput: each setting has runs at about 37 and about 40.5 M/s; the cause was not investigated. (d) Global mutex at 2 readers, paced: [14.26–16.13] M/s.
- Five repetitions of 3–5 s windows. "Longest stall" is a maximum over hundreds of millions of samples and is sensitive to a single preemption; it is reported per run (median and worst), not averaged. Its floor in this session, about 140–160 µs, is the machine's.
- Throughput includes two clock reads per operation (about a third of the cycles of an uncontended read).
- Counters, CPU utilization, context switches and peak resident set come from profiled runs (2 repetitions), not from the timing runs; they are per process, not per thread, and cannot isolate the readers. Kernel frames in the flame graphs have no symbols (`kptr_restrict = 1`).
- The cleaner thread is not pinned.
- `perf c2c` samples loads; a sample count is evidence of where contention is, not a measurement of its cost.
- The temperature rose from 43 to 88 °C during the campaign; repetitions are the outer loop, so drift is spread over all cells, and the ranges are narrow, but thermal limiting cannot be excluded.
- Track R has no concurrent writer by construction, an 80 % hit rate and an uncontrolled lock placement; it is a compatibility reference, not the D.1 measurement.
- The join uses one Snapshot per store (two handles): each table is consistent, the pair is not an atomic cross-store snapshot.
- RCU and the lock stores differ in storage and update strategy, not only in synchronization.
- With reclamation disabled the window is 1 s; its throughput is not directly comparable with the 3 s runs.

8. Raw data

What	Where (under `results/part_d/unwrapped-20261013T115839-fcd2938f49b6/`)
Machine, toolchain, build flags, executable hashes, prerequisites	`env/`
Conditions at 68 checkpoints	`conditions.log`
Timing track. D.1 track C: one CSV row per run (with the lock offset); copy-on-write and reclamation counters	`d1/concurrent/runs.csv`, `cow.csv`
Latency histograms, one per run	`d1/concurrent/hist/`, `d2/*/hist/`, `d3/*/hist/`
Command line, `wrapper: none`, start, end and exit status of every run	`d*/*/stdout/`

What	Where (under `results/part_d/unwrapped-20261013T115839-fcd2938f49b6/`)
D.1 track R	`d1/reference/`
D.2 workloads A and B	`d2/steady/`, `d2/backlog/`
D.3 transactions, join, copy-on-write, generator paths	`d3/txn/`, `d3/join/`, `d3/cow/`, `d3/generator/`
Profiling track. The same cells under perf stat (counters, peak RSS, their own CSVs and histograms)	`profiling/perf_stat/`
Flame graphs (14 SVGs), folded stacks, compressed `perf script` output, commands	`profiling/flamegraphs/`
perf c2c reports (9)	`profiling/c2c/`
Summary tables, CSVs, profiler effects, profile regrouping	`summary/`
Figures (12)	`figures/`

Also under `results/part_d/`: `summary/` and `figures/` (copies of the above, embedded here), `summary/original_vs_corrected.md`, the two superseded campaigns, and `README.md`. Diagnostics outside the campaigns: `results/part_d_diagnostics/`.

Reproduce: `scripts/run_part_d.sh check`, then `scripts/run_part_d.sh all` (about an hour; a new directory each time; refuses to run without mains power, the `performance` governor, usable perf and an idle machine), then `scripts/publish_part_d.sh <campaign directory>`, `scripts/render_part_d_report.py` and `scripts/build_part_d_report.sh`.